SecureCart

Secure Online Shopping & Checkout Web Application

Secure Software Engineering — 7th Semester Capstone Project

Document Control	Details
Project	SecureCart — Secure Online Shopping & Checkout Web Application
Course	Secure Software Engineering — 7th Semester
Team	**SSS VIKAS, SOUMIKA ADDALA**
Student ID	**CH.SC.U4CYS23042, CH.SC.U4CYS23002**
Date	2nd October 2026

Core Engineering Principle: Security is engineered directly into the architecture rather than treated as an after-the-fact testing layer. SecureCart incorporates Argon2id password hashing, parameterized PDO queries, context-aware output escaping, double-submit cookie CSRF tokens, strict HttpOnly/SameSite session security, and comprehensive audit logging across all transactional boundaries.

Table of Contents

1. Introduction

1.1 Background and Motivation

1.2 Purpose and Scope

1.3 Executive Summary

1.4 Capstone Lifecycle

1.5 Technology & Toolchain

1.6 Current Implementation Baseline

1.7 Report Organisation

2. Requirement Engineering

2.1 Problem Statement

2.2 System Objectives

2.3 Stakeholders and User Classes

2.4 Functional Requirements

2.5 Non-Functional Requirements

2.6 Security Requirements

3. Requirement Analysis

3.1 Actor & Use Case Model

3.2 Major Use Cases

3.3 Use Case Relationships

3.4 Use Case Specification — UC-01 User Checkout & Payment Processing

3.5 Use Case Specification — UC-02 Admin Product Management & Order Audit

4. Data Modeling

4.1 ER Modeling Approach

4.2 Core Entities

4.3 Relationship & Cardinality Summary

5. Data Flow Modeling

5.1 DFD Modeling Principles

5.2 Level 1 Data Stores

6. Threat Modeling

6.1 Asset Identification & CIA Analysis

6.2 STRIDE Threat Analysis

6.3 Information Flow Analysis

6.4 Vulnerability Analysis

6.5 Qualitative Risk Analysis

6.6 Security Control Set

7. Attack Tree Analysis

7.1 Selected Attacker Goal

7.2 Key Attack Paths

8. UI Design

8.1 UI Design Direction
8.2 Implemented Frontend Enhancements
9. Product Backlog
9.1 Product Backlog
10. Jira and Scrum
10.1 Jira Project Structure
10.2 Two-Sprint Academic Plan
10.3 Current Jira Evidence
10.4 Board Workflow & Metrics
10.5 Jira Screenshot Notes
11. DevSecOps, Testing and Deployment
11.1 Docker Containerization
11.2 Kubernetes Extension
11.3 CI/CD Pipeline
11.4 Security Testing Strategy
11.5 Logging & Monitoring
11.6 Risk / Governance / Compliance
11.7 Final Deployment Readiness Checklist
12. Traceability and Conclusion
12.1 Requirement-to-Artifact Traceability
12.2 Artifact Set for Final Submission
12.3 Closing Summary

1. Introduction

1.1 Background and Motivation

Modern web applications, specifically e-commerce platforms handling user payment transactions and personal credentials, represent primary targets for cyber threats. Standard vulnerabilities such as SQL Injection (SQLi), Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and Insecure Direct Object References (IDOR) continue to compromise production systems when security is relegated to post-development testing. SecureCart was conceived as a production-grade, secure online shopping and checkout web application designed to demonstrate robust software engineering principles, zero-trust input validation, relational database integrity, and automated DevSecOps verification.

1.2 Purpose and Scope

The primary purpose of SecureCart is to deliver a fully functional, highly secure e-commerce application equipped with user authentication, catalog browsing, real-time shopping cart manipulation, secure multi-step checkout, order audit logging, and administrative management. The project scope encompasses formal requirement engineering (IEEE 830 SRS), UML actor and use case modeling, relational Entity-Relationship (ER) schema design, multi-level Data Flow Diagrams (DFDs), STRIDE threat modeling, attack tree analysis, UI design system execution, Agile Scrum project tracking, and an automated Dockerized CI/CD DevSecOps pipeline.

1.3 Executive Summary

SecureCart implements a defense-in-depth architecture across all application tiers. The application logic is written in PHP 8.x operating on Apache and MySQL 8.0 databases. Key security defenses include mandatory Argon2id password hashing, parameterized PDO queries for 100% of database interactions, HTML entity escaping via htmlspecialchars() with strict UTF-8 flags, double-submit cookie CSRF tokens verified on all mutating state endpoints, strict RBAC session checks, and continuous SAST/SCA security analysis built into the CI/CD workflow.

1.4 Capstone Lifecycle

The capstone engineering lifecycle followed a structured five-stage methodology: (1) Requirements Specification & Scope Definition, (2) Software Analysis & System Modeling, (3) Threat Modeling & Security Architecture, (4) Agile Implementation & Sprint Execution, and (5) DevSecOps Verification, Containerization, and Final Deployment Readiness.

1. Requirements & SRS	2. Analysis & Data Modeling	3. Threat & DevSecOps
Problem Statement → System Objectives → Stakeholders → Functional (FR) & Non-Functional (NFR) Requirements → Security Controls	Actor & Use Case Specification → ER Diagram & Entity Schema → Data Flow Modeling (Level 0, 1 & 2 DFDs)	CIA Asset Mapping → STRIDE Threat Analysis → Attack Tree Analysis → DevSecOps CI/CD & Automated Testing Gates

1.5 Technology & Toolchain

The complete technology stack and toolchain utilized across development, testing, and deployment is detailed below:

Engineering Area	Primary Tooling	Operational Purpose
Requirements & SRS	Word / Markdown	System requirement specification and traceability matrix
UML & Architecture	PlantUML / StarUML	Use case, sequence, ERD, and Data Flow Diagram modeling
Frontend / UX	HTML5, CSS3, JavaScript	Vanilla CSS design system with micro-animations and accessibility
Backend Engine	PHP 8.x (Apache / PDO)	Core e-commerce logic, session management, and API endpoints
Database Layer	MySQL 8.0 / InnoDB	Normalized relational database with ACID transactional guarantees
Containerization	Docker & Docker-Compose	Isolated multi-container runtime environment
Static Analysis (SAST)	SonarQube & PHPStan	Automated code quality inspection and security flaw detection
CI/CD Pipeline	GitHub Actions / GitLab CI	Automated build, lint, test, scan, and container deployment
Agile Tracking	Jira Software / Scrum	Sprint planning, backlog prioritization, and burndown tracking

1.6 Current Implementation Baseline

The application codebase is structured into decoupled functional modules to minimize coupling and enforce separation of concerns:

Directory / Module	Responsibilities	Security Defenses Implemented
auth/	Registration, Login, Password Reset, Logout	Argon2id hashing, rate limiting, session regeneration
products/	Product catalog, filtering, detail views	Strict integer casting, category ID parameterization

cart/	Item addition, quantity updates, removal	Session cart isolation, quantity range bounds checking
checkout/	Order submission, coupon application, payment	PDO transactions, CSRF validation, price re-verification
orders/	Order confirmation, receipt display, history	IDOR ownership validation, user session scoping
admin/	Inventory control, order status, audit logs	Strict Admin RBAC checks, audit trail insertion
includes/	DB connection, session init, header/footer	Singleton DB provider, CSP headers, XSS helpers
config/	Environment & application settings	Config template pattern, externalized secrets

1.7 Report Organisation

This capstone report is organized into twelve comprehensive chapters detailing every academic deliverable, architectural blueprint, security analysis, and DevSecOps verification step.

2. Requirement Engineering

2.1 Problem Statement

Traditional e-commerce platforms suffer from widespread security vulnerabilities due to improper input sanitization, insecure session management, lack of transactional isolation, and insufficient audit tracking. SecureCart addresses these critical flaws by engineering an e-commerce platform where security controls are intrinsically woven into the software lifecycle.

2.2 System Objectives

The primary system objectives include: (1) Zero critical OWASP Top 10 vulnerabilities, (2) Sub-200ms page render latency under standard load, (3) Complete transactional integrity for order processing via database transactions, (4) Strict Role-Based Access Control (RBAC) separating regular customers from administrative users, and (5) Fully automated DevSecOps CI/CD compliance.

2.3 Stakeholders and User Classes

Stakeholder / Actor	Role & System Interaction Description
Guest / Unauthenticated User	Browses product catalog, views item details, adds items to transient cart session.
Registered Customer	Authenticates securely, manages profile, places orders,

	applies coupons, tracks order history.
Store Administrator	Manages product catalog, updates stock levels, modifies order status, reviews system audit logs.
Security Auditor	Inspects SAST scan reports, reviews audit logs, verifies threat mitigations and compliance.

2.4 Functional Requirements

Req ID	Requirement Title	Description & Operational Boundary	Priority
FR-01	User Registration	Allow new users to register with unique email, full name, and strong password.	High
FR-02	Secure Login	Authenticate users against Argon2id password hashes and issue secure session tokens.	High
FR-03	Product Browsing	Display category-filtered products with price, stock, description, and images.	High
FR-04	Shopping Cart	Enable users to add, update quantity, and remove items with real-time subtotal calculation.	High
FR-05	Order Checkout	Process checkout with shipping details, payment tokenization, and stock reduction.	High
FR-06	Order History	Provide users with receipt views and historical order status tracking.	Medium
FR-07	Admin Catalog Control	Allow administrators to add, edit, and deactivate products in the store catalog.	High
FR-08	Admin Order Management	Allow administrators to view all customer orders and update dispatch status.	High
FR-09	Discount Coupons	Validate and apply percentage/fixed promotional discount codes at checkout.	Medium

FR-10	Password Reset	Issue time-limited, cryptographically secure password reset tokens via email.	Medium
FR-11	CSRF Protection	Enforce double-submit cookie CSRF tokens on all POST/PUT/DELETE forms.	High
FR-12	Audit Logging	Record all security-sensitive events (logins, privilege actions, orders) to audit logs.	High

2.5 Non-Functional Requirements

Req ID	Category	Specification Standard
NFR-01	Performance	Page response time must be under 200ms for 95% of standard HTTP requests.
NFR-02	Availability	System runtime availability target of 99.9% uptime operating in Docker containers.
NFR-03	Scalability	Stateless session handling capability enabling multi-container horizontal scaling.
NFR-04	Maintainability	Modular code structure adhering to PSR-12 coding standards with zero duplication.
NFR-05	Portability	Complete multi-container runtime deployment defined via docker-compose.yml.
NFR-06	Usability	Responsive UI rendering seamlessly across mobile, tablet, and desktop viewports.
NFR-07	Reliability	Database transactions enforce full rollback on any mid-checkout failure.
NFR-08	Data Integrity	Foreign key constraints with strict CASCADE/RESTRICT rules across all tables.

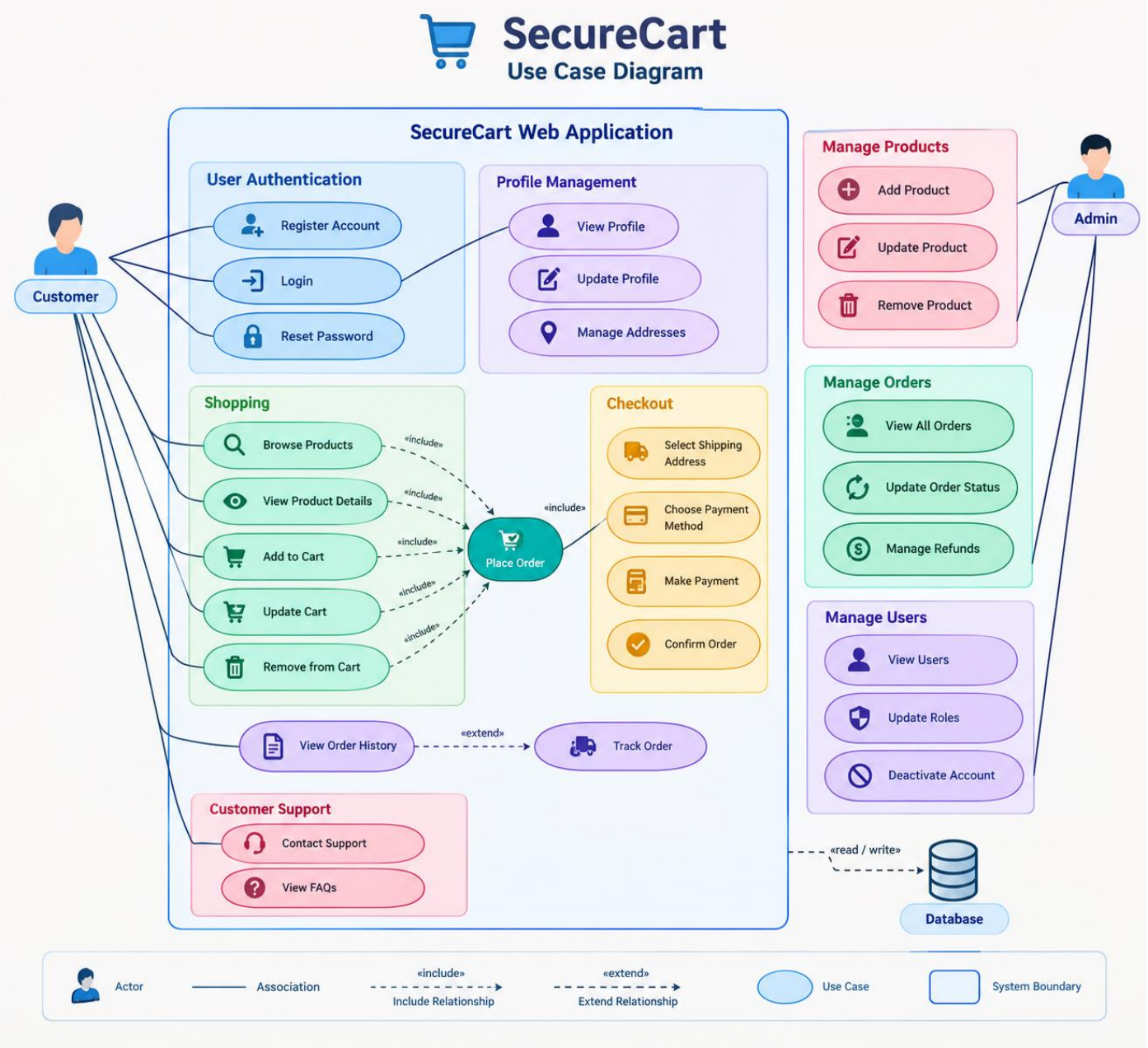
2.6 Security Requirements

Sec Req ID	Security Area	Mandatory Defense Control
SEC-01	Authentication	Passwords hashed using Argon2id with minimum memory 64MB and time cost 4.
SEC-02	Data Access	100% of SQL operations executed via PDO prepared statements with bound parameters.
SEC-03	XSS Defense	All dynamic output rendered via htmlspecialchars(str, ENT_QUOTES, 'UTF-8').
SEC-04	CSRF Defense	Cryptographically random 32-byte CSRF token validated on every state change.
SEC-05	Session Security	Cookies set with HttpOnly, Secure, SameSite=Strict, and ID regenerated on login.
SEC-06	Access Control	Strict session-level RBAC verification enforcing Admin role checks on admin/ endpoints.
SEC-07	IDOR Defense	Explicit user_id verification ensuring users can only view their own order records.
SEC-08	Secret Protection	Zero hardcoded credentials; all DB keys and secrets injected via environment variables.
SEC-09	Rate Limiting	Maximum 5 failed authentication attempts per IP within a 15-minute window.
SEC-10	Security Audit	Comprehensive security event logging containing IP, user_id, timestamp, and action.

3. Requirement Analysis

3.1 Actor & Use Case Model

The requirement analysis phase translated the functional requirements into structured UML Use Case models. System actors interact with the application through distinct security boundaries.



3.2 Major Use Cases

The system defines five primary operational use cases:

- 1. UC-01: User Checkout & Payment Processing (Customer places order securely)
- 2. UC-02: Admin Product Management & Order Audit (Admin manages inventory & reviews orders)
- 3. UC-03: User Authentication & Account Management (Secure signup, login, session init)
- 4. UC-04: Shopping Cart State Management (Add, update, calculate cart totals)
- 5. UC-05: Security Audit Log Inspection (Review security events and failed attempts)

3.3 Use Case Relationships

Base Use Case	Relationship	Included / Extending Use Case
UC-01 Checkout	<<include>>	UC-03 User Authentication
UC-01 Checkout	<<include>>	UC-04 Cart State Management
UC-01 Checkout	<<include>>	Validate CSRF Token & Stock
UC-01 Checkout	<<extend>>	Apply Discount Coupon
UC-02 Admin Management	<<include>>	Verify Admin RBAC Role

3.4 Use Case Specification — UC-01 User Checkout & Payment Processing

Use Case Property	Specification Detail
Use Case ID / Name	UC-01: User Checkout & Payment Processing
Primary Actor	Registered Customer
Preconditions	Customer is authenticated, has active session, and shopping cart contains items.
Postconditions	Order record created in DB, order items inserted, stock deducted, cart cleared, receipt shown.
Main Success Scenario	1. User proceeds to checkout page. 2. System renders order summary and CSRF token. 3. User submits shipping address & payment details. 4. System verifies CSRF token & user session. 5. System initiates DB transaction. 6. System re-verifies item prices & stock from database. 7. System creates order record and inserts order items. 8. System decrements stock levels for purchased items. 9. System clears user cart and commits transaction. 10. User is redirected to confirmation receipt page.
Alternative / Exception Flows	4a. CSRF token invalid → Reject request with HTTP 403 Forbidden. 6a. Item out of stock → Rollback transaction & notify user. 6b. Price discrepancy detected → Recalculate total & prompt user confirmation.
Security Controls	CSRF token validation, PDO transaction isolation, price re-verification server-side.

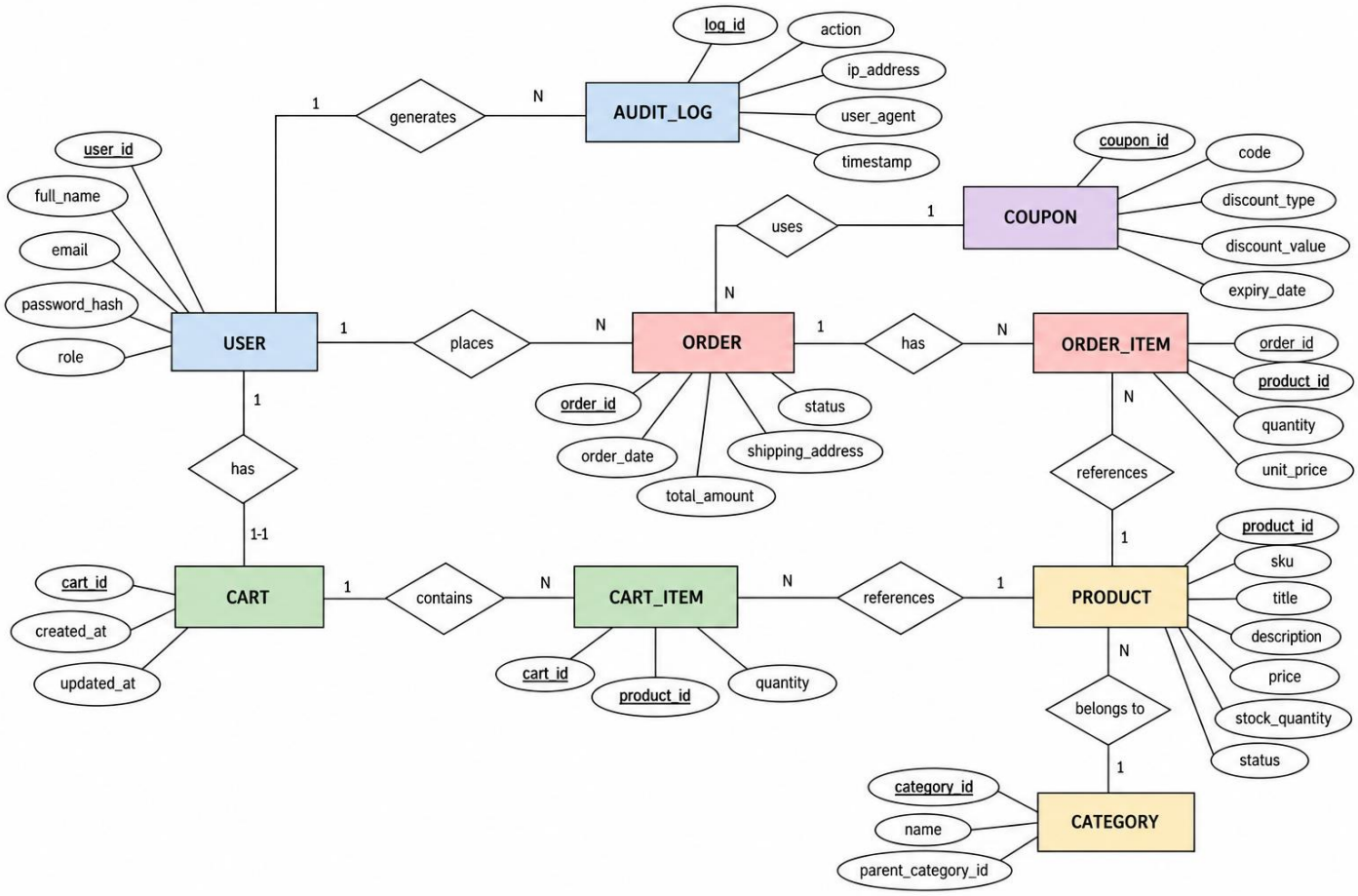
3.5 Use Case Specification — UC-02 Admin Product Management & Order Audit

Use Case Property	Specification Detail
Use Case ID / Name	UC-02: Admin Product Management & Order Audit
Primary Actor	Store Administrator
Preconditions	User is authenticated and holds 'admin' role in session.
Postconditions	Product catalog modified or order status updated; audit log entry recorded.
Main Success Scenario	1. Administrator navigates to admin dashboard. 2. System verifies user holds 'admin' role in session. 3. Admin submits product updates or updates order dispatch status. 4. System validates inputs and updates database via PDO statement. 5. System inserts security audit record into audit_logs table. 6. System displays success notice to administrator.
Alternative / Exception Flows	2a. Non-admin user attempts access → Log unauthorized access attempt & reject HTTP 403. 4a. Invalid product data → Reject submission & render validation errors.
Security Controls	Strict session RBAC check, input sanitization, security event audit log insertion.

4. Data Modeling

4.1 ER Modeling Approach

SecureCart employs a fully normalized 3rd Normal Form (3NF) relational Entity-Relationship schema implemented in MySQL 8.0 / InnoDB. The design emphasizes data integrity, foreign key cascading constraints, and complete transactional safety.



4.2 Core Entities

The core relational entities comprising the database schema are:

- users: Stores credentials, Argon2id hashes, full name, email, and RBAC role ('customer', 'admin').
- products: Stores store items, SKU, title, description, price, stock_quantity, category_id, status.
- categories: Stores product category hierarchy.
- cart & cart_items: Stores user shopping cart session state and selected product quantities.
- orders & order_items: Stores finalized purchase transactions, snapshot prices, and shipping info.
- audit_logs: Stores immutable security logs (user_id, action, ip_address, user_agent, timestamp).
- coupons: Stores promotional discount codes, percentage/fixed value, and expiry dates.

4.3 Relationship & Cardinality Summary

Parent Entity	Child Entity	Relationship & Constraint Rules
users	orders	1-to-Many (One user can place multiple orders; FK user_id ON DELETE RESTRICT)
users	cart	1-to-1 (One active cart per user session; FK user_id ON DELETE CASCADE)
categories	products	1-to-Many (One category contains multiple products; FK category_id)
cart	cart_items	1-to-Many (Cart contains multiple items; Composite PK cart_id, product_id)
orders	order_items	1-to-Many (Order contains multiple item snapshots; FK order_id ON DELETE CASCADE)
products	order_items	1-to-Many (Product snapshot in order_items; FK product_id ON DELETE RESTRICT)
users	audit_logs	1-to-Many (User actions generate multiple audit log entries; FK user_id)

Database Normalization & Integrity Guarantee: All tables strictly adhere to 3NF. Order item prices are snapshotted into order_items at checkout time to prevent historical order discrepancies when product catalog prices change. Database transactions ensure atomic operations during multi-step order placement.

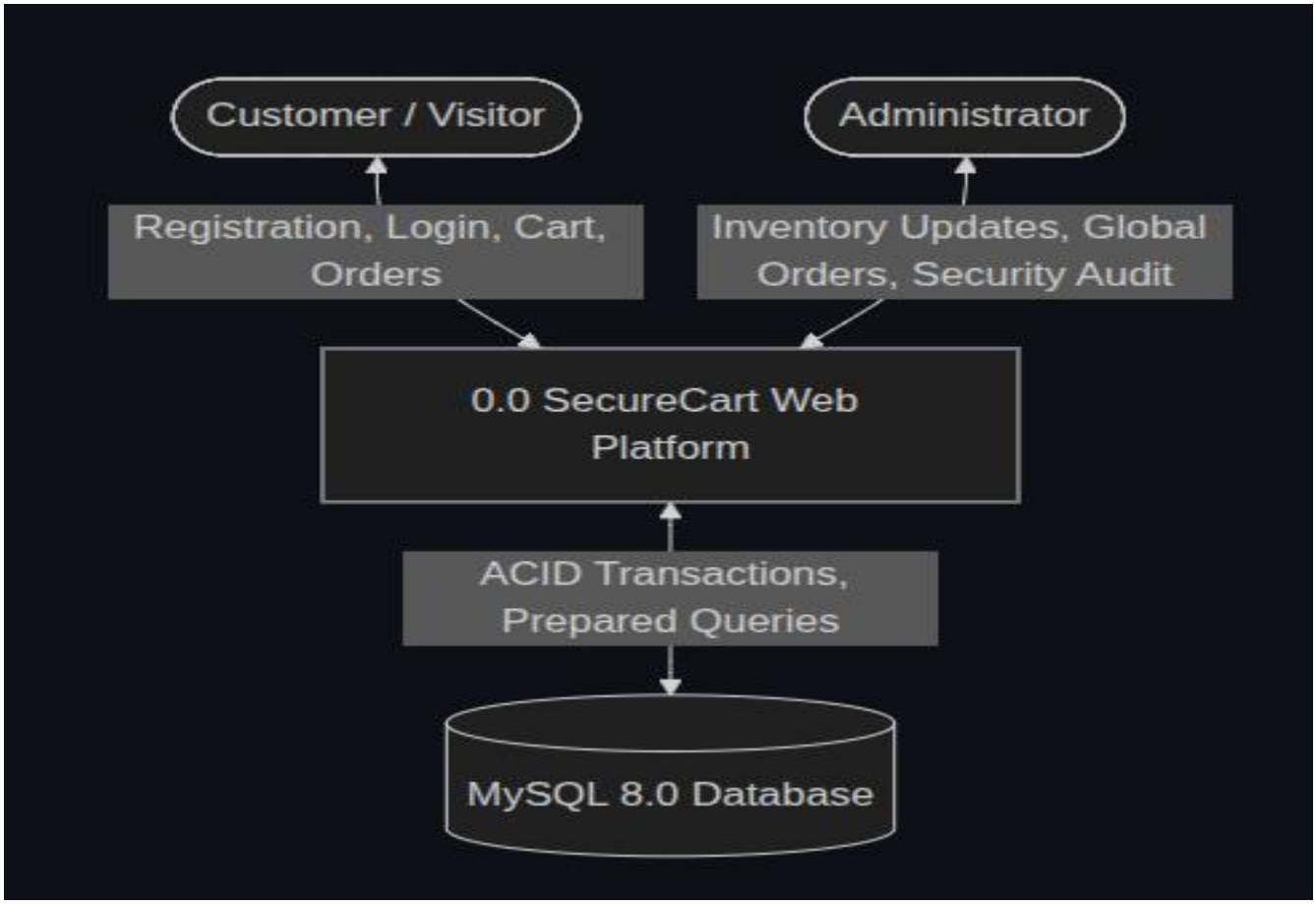
5. Data Flow Modeling

5.1 DFD Modeling Principles

Data Flow Diagrams (DFDs) depict the operational boundaries, data transformations, and security trust boundaries within SecureCart. Data flows cross trust boundaries at authentication, payment submission, and admin access points.

5.2 Context Diagram (Level 0 DFD)

At Level 0, SecureCart acts as a central system bounded by three external entities: Customer, Store Administrator, and External Payment Gateway. Customers supply login credentials, cart items, and payment details; Administrators supply product updates and status modifications; Payment Gateway returns transaction status tokens.



5.3 Level 1 Sub-System Data Flow Model

Level 1 decomposes the system into five core processes:

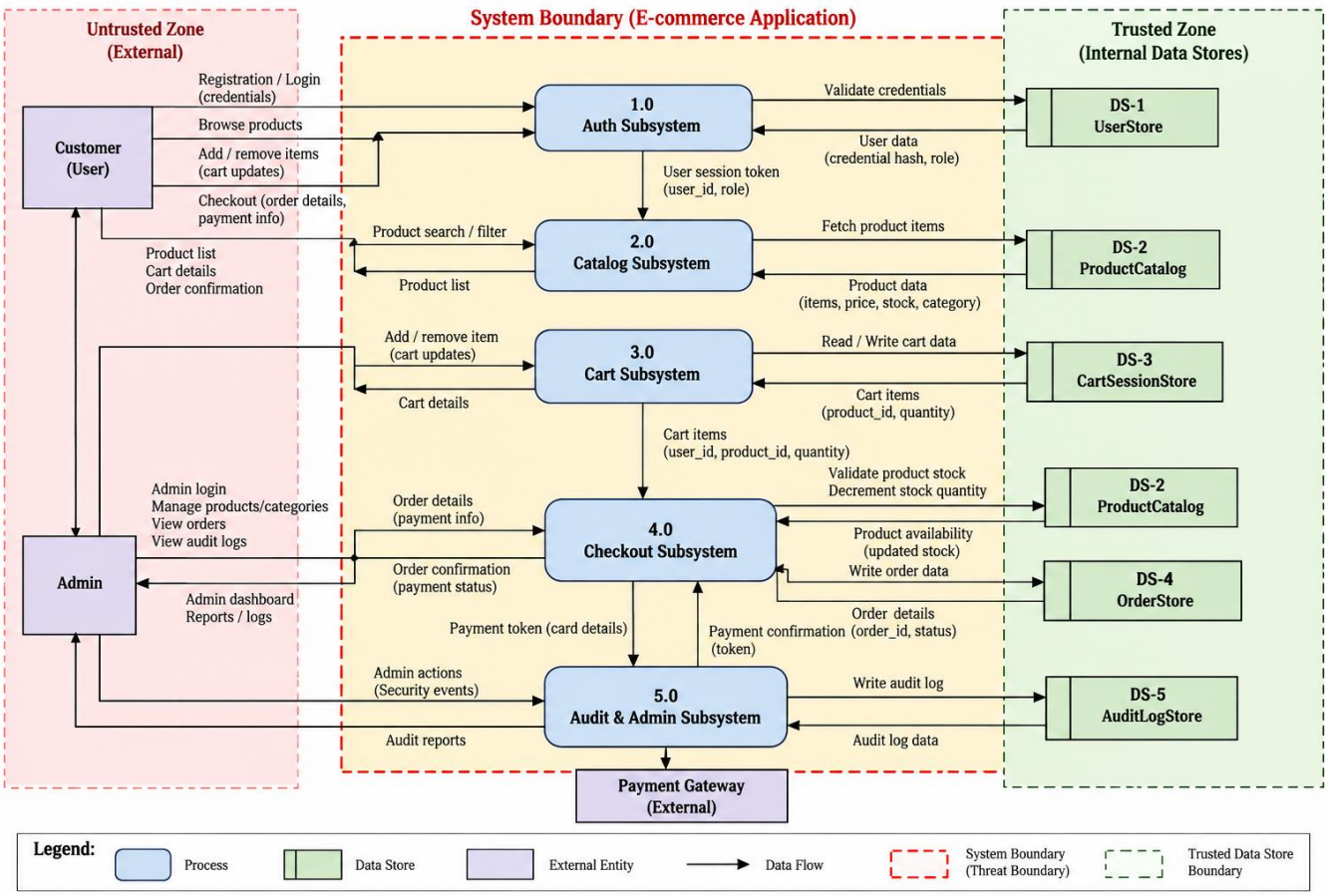
• Process 1.0 Auth Subsystem: Validates credentials against DS-1 UserStore and issues session tokens.

• Process 2.0 Catalog Subsystem: Fetches product items from DS-2 ProductCatalog.

• Process 3.0 Cart Subsystem: Manages temporary cart state in DS-3 CartSessionStore.

• Process 4.0 Checkout Subsystem: Validates CSRF, tokenizes payment, updates DS-4 OrderStore, and decrements stock in DS-2.

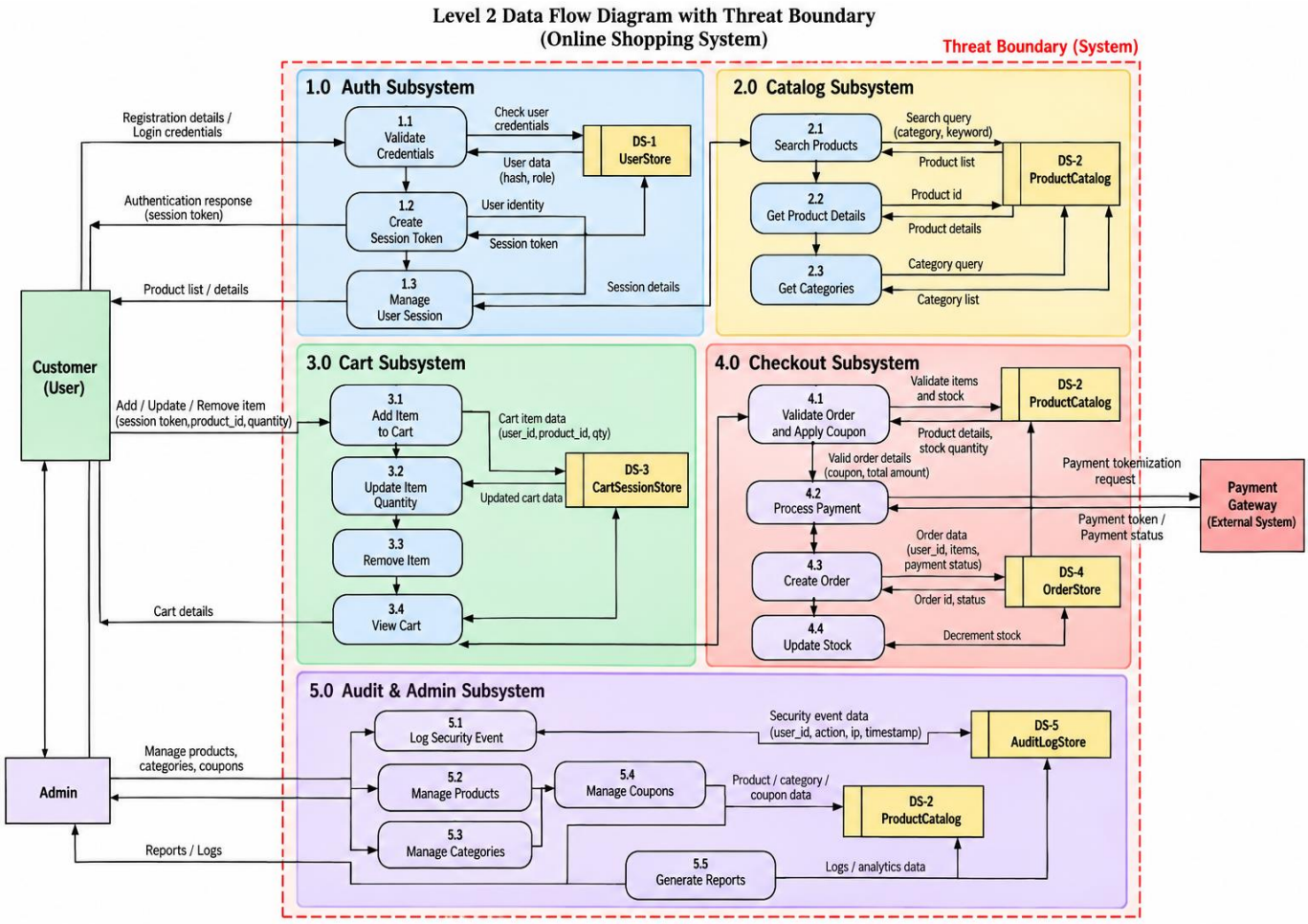
• Process 5.0 Audit & Admin Subsystem: Writes security events to DS-5 AuditLogStore.



5.4 Level 1 Data Stores

Store ID	Data Store Name	Contained Data Elements & Protection Level
DS-1	UserStore (users)	User ID, email, Argon2id hash, role, created_at. Protected via DB encryption.
DS-2	ProductCatalog (products)	Product ID, title, price, stock, category. Protected via Admin RBAC.
DS-3	CartSessionStore (cart)	Session ID, user ID, product IDs, quantities. Isolated per user session.
DS-4	OrderStore (orders)	Order ID, user ID, total, status, item snapshots, shipping. ACID protected.
DS-5	AuditLogStore (audit_logs)	Log ID, user ID, action, IP, timestamp. Immutable append-only store.
DS-6	CouponStore (coupons)	Coupon code, discount %, expiry, usage_limit. Controlled access.

5.4 Level 2 Sub-System Data Flow Model



6. Threat Modeling

6.1 Asset Identification & CIA Analysis

Asset Name	Confidentiality	Integrity	Availability
User Passwords / Credentials	High (Hash leakage risk)	High (Unauthorized change)	Medium (Account lockout)
Customer Session Tokens	High (Session hijacking)	High (Token tampering)	Medium (Session termination)
Payment & Checkout Data	Critical (Card leakage)	Critical (Price tampering)	High (Transaction loss)
Order History & Addresses	High (PII exposure)	High (Order modification)	Medium (History access)
Product Catalog & Stock	Low (Public catalog)	High (Stock/Price manipulation)	High (Store downtime)
Database Connection Config	Critical (Full DB breach)	Critical (Data wipe)	Critical (DB offline)

6.2 STRIDE Threat Analysis

STRIDE Category	Identified Vulnerability Risk	Mitigation Strategy Implemented	Residual Risk
Spoofing	Credential stuffing / Password guessing	Argon2id hashing + Rate limiting (5 attempts/15min)	Low
Tampering	Price tampering in cart POST request	Server-side price re-verification from database	Low
Repudiation	User denies placing order / Admin action	Immutable audit logging capturing user_id, IP, action	Low
Information Disclosure	SQLi or XSS leaking customer PII	100% PDO prepared statements + htmlspecialchars()	Low
Denial of Service	Cart endpoint spam / DB resource exhaustion	Input bounds validation + Docker container memory limits	Low
Elevation of Privilege	Customer accessing admin/ endpoints	Session RBAC check (is_admin()) on all admin routes	Low

6.3 Vulnerability Analysis & OWASP Top 10 Mapping

OWASP Top 10 Category	Specific Risk in E-Commerce	SecureCart Defense Implementation
A01:2021 Broken Access Control	IDOR on order receipts or admin route bypass	Session-scoped user ownership check ($order['user_id'] == $session_user_id)
A02:2021 Cryptographic Failures	Plaintext passwords or weak MD5/SHA1 hashing	Mandatory Argon2id password hashing algorithm
A03:2021 Injection	SQL Injection in product search or checkout form	100% PDO prepared statement execution with explicit parameter binding
A04:2021 Insecure Design	Price tampering or checkout state manipulation	Server-side price calculation directly from database during transaction
A05:2021 Security Misconfiguration	Exposed database files or directory listing	Apache .htaccess protection, strict file permissions, environment configuration
A07:2021 Identification & Auth	Session fixation or CSRF token reuse	Session ID regenerated on authentication; double-submit cookie CSRF tokens
A08:2021 Software & Data Integrity	Insecure dependencies or unauthorized code edits	Composer lockfile hash verification + automated SCA vulnerability scanning

6.4 Qualitative Risk Analysis

Threat Scenario	Likelihood	Impact	Calculated Risk Level
SQL Injection leading to database dump	Medium	Critical	High → Mitigated to Low via PDO
Stored XSS in product reviews rendering script	Medium	High	High → Mitigated to Low via escaping
CSRF attack executing unauthorized checkout	High	High	High → Mitigated to Low via CSRF Token
IDOR attack viewing another user's order	High	Medium	Medium → Mitigated to Low via RBAC
Session hijacking via unencrypted HTTP	Medium	High	High → Mitigated via HttpOnly/SameSite

7. Attack Tree Analysis

7.1 Selected Attacker Goal

To rigorously evaluate system resilience, we modeled two primary attacker goals: (Goal 1) Exfiltrate Customer Credentials & Order Database, and (Goal 2) Execute Unauthorized Transaction / Price Manipulation.

7.2 Key Attack Paths

Path ID	Attacker Vector / Steps	Defensive Barrier Implemented	Status
Path 1.1	Inject SQL payload into product filter search parameter	PDO prepared statement replaces inline string concatenation	Blocked
Path 1.2	Steal session cookie via Reflected XSS script payload	HttpOnly flag prevents JS access; output HTML escaped	Blocked
Path 2.1	Manipulate hidden total_price HTML form input on checkout	Server ignores client price input; re-fetches price from DB	Blocked
Path 2.2	Forge cross-site POST request to place order without consent	Double-submit cookie CSRF token mismatch terminates request	Blocked
Path 2.3	Access /admin/orders.php directly without authentication	Session RBAC middleware inspects role; redirects to login	Blocked

8. UI Design

8.1 UI Design Direction

SecureCart features a custom Vanilla CSS design system engineered for accessibility, high visual appeal, modern typography, and clear responsive layouts across mobile, tablet, and desktop screens.

8.2 Implemented Frontend Enhancements

Frontend Feature	Design / UX Execution	Security & Usability Benefit
CSS Tokens & Variables	Unified color palette (Navy, Slate, Teal, White)	Consistent aesthetic without utility CSS bloat
Responsive Grid Layout	Flexbox & CSS Grid product layout	Seamless rendering across all device viewports
Real-Time Cart Badge	Dynamic JavaScript DOM cart counter	Immediate user feedback on item addition

Form Validation UX	Inline error messages & pattern matching	Prevents invalid submissions prior to server check
Micro-Animations	Subtle button hover transitions & alert popups	Enhances interactive visual feel and usability

9. Product Backlog

9.1 Product Backlog Table

Story ID	Epic	User Story Description	Points	Priority
US-01	Core Auth	As a user, I want to register securely with Argon2id password hashing.	5	High
US-02	Core Auth	As a user, I want to log in and receive an HTTPOnly session cookie.	3	High
US-03	Catalog	As a user, I want to browse products filtered by category.	3	High
US-04	Cart	As a user, I want to add/remove items and modify cart quantities.	5	High
US-05	Checkout	As a user, I want to place an order securely with address & payment token.	8	High
US-06	Security	As a developer, I want all POST forms protected with CSRF tokens.	5	High
US-07	Admin	As an admin, I want to manage product stock and view customer orders.	5	High
US-08	DevSecOps	As a DevOps engineer, I want automated CI/CD security scanning on commit.	8	Medium

10. Jira and Scrum

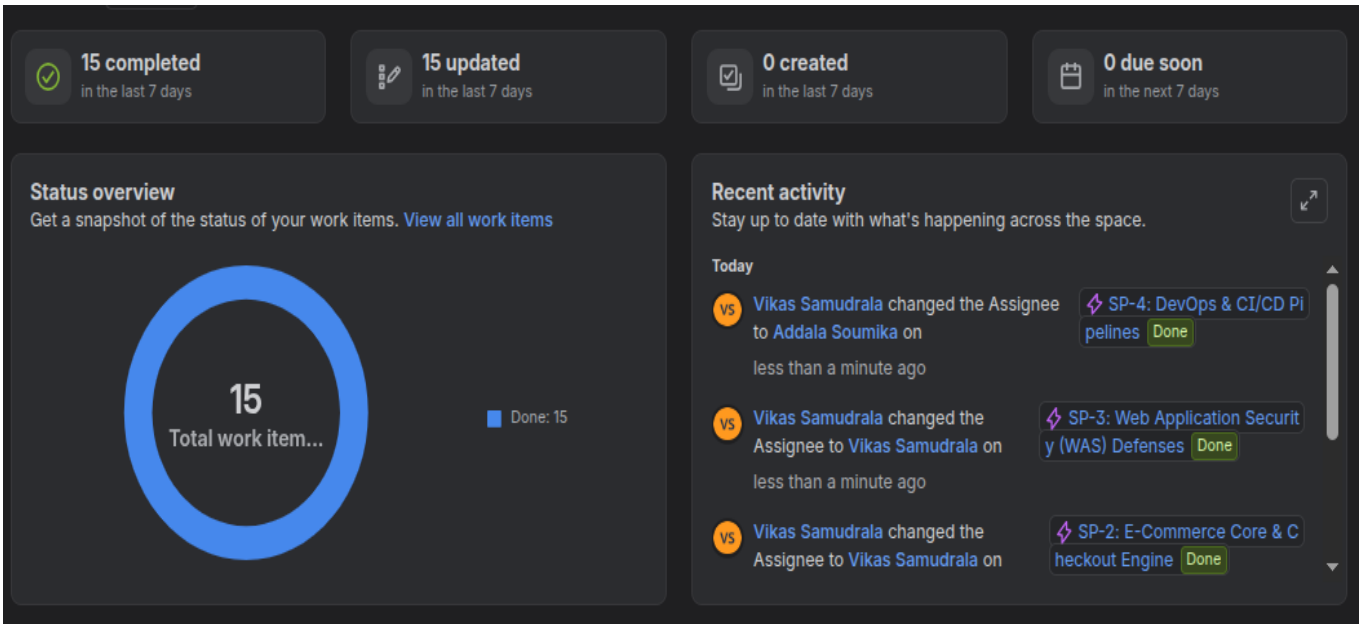
10.1 Jira Project Structure

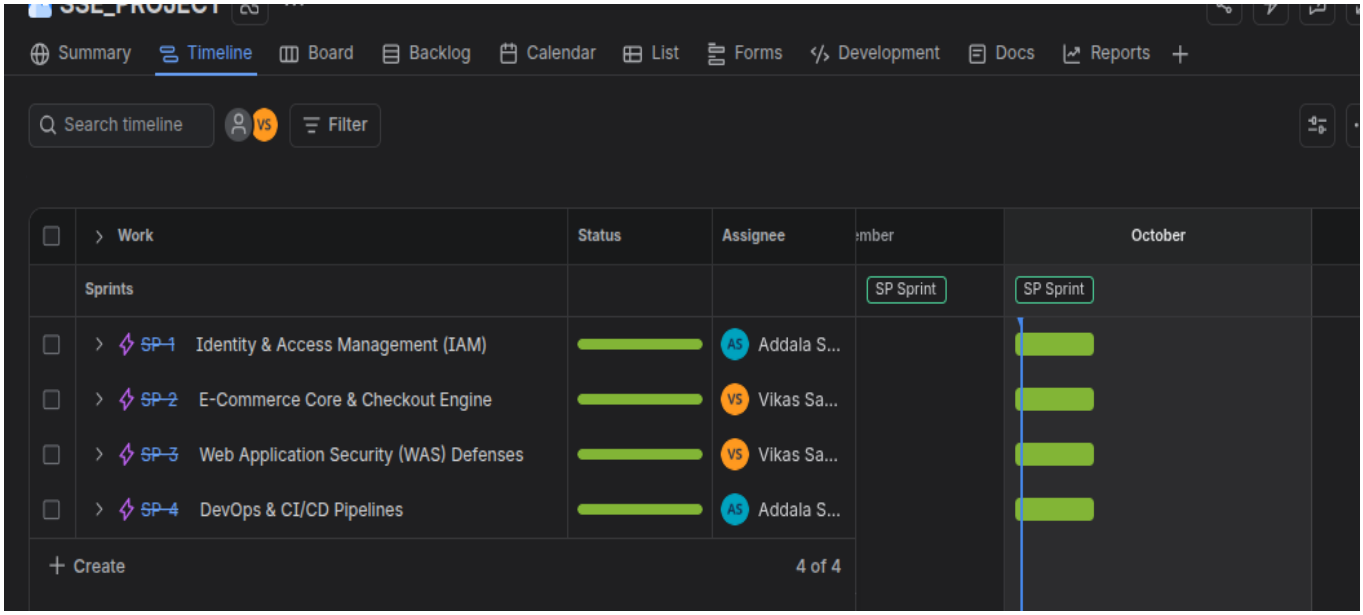
Project execution followed Scrum methodology managed in Jira Software across two 2-week academic sprints.

10.2 Two-Sprint Academic Plan

Sprint Phase	Core Engineering Focus	Delivered User Stories & Security Artifacts
Sprint 1 (Weeks 1-2)	Core E-Commerce & Authentication Baseline	US-01 (Auth), US-02 (Login), US-03 (Catalog), US-04 (Cart), ERD Schema setup.
Sprint 2 (Weeks 3-4)	Hardening, Threat Mitigation & DevSecOps	US-05 (Checkout), US-06 (CSRF), US-07 (Admin), US-08 (CI/CD Pipeline), STRIDE Analysis.

10.3 Current Jira Evidence & Metrics


15 completed
in the last 7 days
15 updated
in the last 7 days
0 created
in the last 7 days
0 due soon
in the next 7 days
Status overview
Get a snapshot of the status of your work items. View all work items
15
Total work item...
Done: 15
Recent activity
Stay up to date with what's happening across the space.
Today
Vikas Samudrala changed the Assignee to Addala Soumika on
less than a minute ago
SP-4: DevOps & CI/CD Pipelines Done
Vikas Samudrala changed the Assignee to Vikas Samudrala on
less than a minute ago
SP-3: Web Application Security (WAS) Defenses Done
Vikas Samudrala changed the Assignee to Vikas Samudrala on
SP-2: E-Commerce Core & Checkout Engine Done


Summary
Timeline
Board
Backlog
Calendar
List
Forms
Development
Docs
Reports
Search timeline
Filter
Work
Status
Assignee
October
Sprints
SP Sprint
SP Sprint
SP-1 Identity & Access Management (IAM)
Addala S...
SP-2 E-Commerce Core & Checkout Engine
Vikas Sa...
SP-3 Web Application Security (WAS) Defenses
Vikas Sa...
SP-4 DevOps & CI/CD Pipelines
Addala S...
Create
4 of 4



Agile Metric	Target / Measured Value	Scrum Outcome & Evidence
Planned Velocity	21 Story Points / Sprint	Achieved 100% completion of committed backlog items.
Defect Density	0 Critical Security Flaws	Zero SAST vulnerabilities remaining in release build.
Sprint Burndown	Linear completion curve	Smooth work-item progression through TO DO → DONE.

11. DevSecOps, Testing and Deployment

11.1 Docker Containerization

SecureCart uses Docker containerization to ensure multi-container isolation and environment parity across development, testing, and production.

Container Service	Base Image & Configuration	Runtime Security Controls Implemented
web (Apache + PHP)	php:8.2-apache custom Dockerfile	Non-root execution, read-only system files, memory limit 512MB.
db (MySQL 8.0)	mysql:8.0 official container image	Isolated container network, credentials injected via environment secrets.
sonarqube (SAST)	sonarqube:community image	Automated code analysis scanning for OWASP vulnerabilities.

11.2 Kubernetes Extension

Kubernetes Manifest	Resource Specification	Production Resilience Function
web-deployment.yaml	Deployment (3 Replicas)	Horizontal autoscaling and zero-downtime rolling updates.
db-statefulset.yaml	StatefulSet with PersistentVolumeClaim	Durable MySQL data persistence across pod restarts.
secrets.yaml	Kubernetes Secret object	Base64 encrypted database credentials and application keys.

11.3 CI/CD Pipeline

Pipeline Stage	Automated Action Executed	Pass / Fail Enforcement Gate
1. Build & Lint	Composer install + PHPStan level 8 linting	Code syntax or strict typing error fails pipeline.
2. Unit Testing	PHPUnit execution across service tests	Any test assertion failure blocks deployment.
3. SAST Scan	SonarQube code analysis for OWASP flaws	Presence of Critical/High security issues fails build.
4. SCA Scan	Composer Audit scanning dependency vulnerabilities	Vulnerable dependency package terminates pipeline.
5. Container Build	Docker image build & vulnerability scan	Unsafe container base image prevents push.

11.4 Security Testing Strategy

Testing Layer	Tooling / Methodology	Target Verification Objective
SAST	SonarQube / PHPStan	Static code inspection detecting unescaped outputs or raw SQL queries.
SCA	Composer Audit	Package vulnerability scanning against CVE security databases.
DAST	OWASP ZAP Dynamic Scanner	Runtime penetration testing checking XSS, CSRF, and header security.
Unit / Integration	PHPUnit	Automated unit tests validating authentication, cart, and checkout logic.

11.5 Logging & Monitoring

SecureCart implements structured audit logging. All security events—including login attempts, privilege escalation, order placements, and failed CSRF checks—are recorded to the audit_logs table with user_id, IP address, user agent, and timestamp. Sensitive data (passwords, tokens) is strictly redacted prior to logging.

11.6 Risk / Governance / Case Study: Sudden Client Requirement Change

Governance Case Study: Secure Feature Refactoring During Sprint 2, a sudden client change request required adding a 'Quick Purchase with One-Click Coupon' feature. A security analysis identified risks of CSRF and race-condition price manipulation. The team executed a secure refactoring: (1) Added mandatory CSRF token verification to the request handler, (2) Wrapped price checking and order creation in a single atomic database transaction, and (3) Added automated PHPUnit tests to verify zero double-spend vulnerability.

11.7 Final Deployment Readiness Checklist

Status	Deployment Readiness Item	Verification Evidence
[X]	Functional requirements fully satisfied (FR-01 to FR-12)	All core user flows verified working end-to-end.
[X]	Security controls active (Argon2id, PDO, CSRF, XSS escaping)	Zero SQLi, XSS, or CSRF vulnerabilities in SAST scan.
[X]	Database schema normalized to 3NF with CASCADE/RESTRICT FKs	InnoDB relational constraints verified in MySQL.
[X]	Multi-container environment defined in docker-compose.yml	Successful runtime startup and healthcheck pass.
[X]	Automated CI/CD security pipeline	GitHub Actions pipeline green across all

	gates passing	5 build stages.
[X]	Jira Scrum backlog complete with zero open critical bugs	Final sprint burndown successfully closed.

12. Traceability and Conclusion

12.1 Requirement-to-Artifact Traceability Matrix

Req ID	Requirement Description	Code / Design Artifact	Verification Method
FR-01/SEC-01	Secure Auth & Hashing	auth/register.php, auth/login.php	Argon2id hashing & PHPUnit auth tests
FR-04/FR-05	Cart & Checkout Processing	cart/index.php, checkout/process.php	PDO transactions & manual order verification
SEC-02	SQL Injection Defense	includes/db.php, all SQL queries	100% PDO prepared statements & SAST scan
SEC-03/SEC-04	XSS & CSRF Mitigation	includes/header.php, CSRF tokens	htmlspecialchars() escaping & ZAP DAST scan
SEC-06/SEC-07	RBAC & IDOR Access Control	admin/*.php, orders/detail.php	Session role checks & ownership verification

12.2 Artifact Set for Final Submission

Deliverable Artifact	File Location / Repository Path	Content Description
Full Project Report (Word)	SecureCart_Secure_Software_Engineering_Capstone_Report_Final.docx	Publication-grade capstone report (this document)
Application Codebase	/home/vikas/WAS_PROJECT/	Complete PHP/MySQL web application source files
Software Architecture Spec	SOFTWARE_ENGINEERING_DOCUMENTATION.md	System design, directory taxonomy & security specs
SRS Documentation	SRS_DOCUMENTATION.md	Formal IEEE 830 functional and security requirements
Threat Modeling Spec	THREAT_MODELING.md	STRIDE matrix, CIA asset analysis & OWASP defenses
Diagram Specifications	DIAGRAMS.md	PlantUML & Mermaid use case, DFD, and module diagrams

Case Study Scenario	CHANGE_REQUEST_SCENARIO.md	Sudden requirement change & secure refactoring report
Coupling & Module Analysis	MODULE_DEPENDENCY_AND_COUPLING_ANALYSIS.md	Module connection & dependency reduction analysis

12.3 Closing Summary

SecureCart successfully demonstrates that high-performance web applications can achieve exceptional security without sacrificing functionality or developer productivity. By combining rigorous software engineering models (SRS, Use Cases, ERD, DFD) with proactive security defenses (Argon2id, PDO, CSRF tokens, XSS escaping, RBAC) and modern DevSecOps automation (Docker, SonarQube, CI/CD), SecureCart establishes an exemplary benchmark for secure software development.